Report_Crypto_Final

Benchmarking Schwaemm128 on FPGA Soft-Core CPUs

VexRiscv and PicoRV32 Implementations in C and Rust

---

1. Introduction

Lightweight authenticated encryption is essential for embedded and IoT systems where computational resources are constrained. The **SPARKLE** permutation and its AEAD scheme **Schwaemm** were finalists in the NIST Lightweight Cryptography (LWC) competition due to their efficient ARX-based design and small hardware footprint.

This project benchmarks **Schwaemm128**, implemented in both **C** and **Rust**, on two FPGA-hosted RISC-V soft-core processors:

- **VexRiscv** – pipelined, performance-oriented
- **PicoRV32** – minimalistic, compact soft-core

To synthesize and implement the processors, **Xilinx Vivado** was used to generate bitstreams for the FPGA. The benchmarking programs were cross-compiled, with Rust builds taking **approximately twice as long as C builds** due to the heavier toolchain and LLVM backend.

The goals of this study are:

1. Measure encryption and decryption performance in cycles and instruction count
2. Compare C vs Rust implementations on the same hardware
3. Compare VexRiscv vs PicoRV32 performance
4. Draw conclusions about Schwaemm128 suitability for lightweight RISC-V systems

---

2. Background

2.1 SPARKLE Permutation

SPARKLE is a family of permutations designed around **Add–Rotate–XOR (ARX)** operations. Its key goals are:

- Efficiency in lightweight CPUs

- Simple hardware implementation
- Security against differential/linear cryptanalysis
- Good performance across both hardware and software

SPARKLE384, used in Schwaemm128, applies a mixture of “big” and “slim” rounds for mixing and diffusion.

2.2 Schwaemm AEAD

Schwaemm128 is an authenticated encryption scheme that uses the SPARKLE384 permutation. Its parameters include:

- **Key:** 128 bits
- **Rate:** 256 bits
- **Capacity:** 128 bits
- **Tag length:** 128 bits

The algorithm absorbs associated data, encrypts plaintext while authenticating it, and produces a tag. Decryption verifies the tag before releasing the plaintext.

Schwaemm is designed to perform well in constrained environments such as FPGA soft-cores.

---

3. Experimental Setup

3.1 Hardware and Toolflow

Both RISC-V cores were synthesized and implemented using **Xilinx Vivado**:

- Vivado Synthesis → Implementation → Bitstream Generation
- UART, memory, and performance counters (cycle + instruction counters) were integrated
- Both cores operated at the same clock frequency for fair comparison

---

3.2 C and Rust Implementations

Two software stacks were benchmarked:

C Implementation

- Compiled using a RISC-V GCC toolchain
- Fast build times
- Small binaries optimized for embedded use

Rust Implementation

- Cross-compiled using `rustc` and a RISC-V target
- Larger binaries and more instructions because of ABI conventions

Both implementations performed:

1. Encryption of a 32-byte message
2. Decryption of ciphertext
3. Authentication tag verification
4. Plaintext comparison
5. Reporting of cycles and instruction counts

---

4. Benchmark Results

Below are the **exact results** from FPGA runs.

4.1 PicoRV32 — C Implementation

Encryption

- **ENC_CYCLES:** 52,105
- **ENC_INSTRS:** 9,267

Decryption

- **DEC_CYCLES:** 50,958
- **DEC_INSTRS:** 9,273

4.2 PicoRV32 — Rust Implementation

Encryption

- **ENC_CYCLES:** 99,224
- **ENC_INSTRS:** 16,488

Decryption

- **DEC_CYCLES:** 100,182
- **DEC_INSTRS:** 16,637

Observation:
Rust takes *almost exactly 2× the cycles* of the C implementation, reflecting both larger instruction count and less-optimized code generation for PicoRV32.

4.3 VexRiscv — C Implementation

Encryption

- **ENC_CYCLES:** 10,834
- **ENC_INSTRS:** 9,267

Decryption

- **DEC_CYCLES:** 10,764
- **DEC_INSTRS:** 9,273

4.4 VexRiscv — Rust Implementation

Encryption

- **ENC_CYCLES:** 24,765
- **ENC_INSTRS:** 16,515

Decryption

- **DEC_CYCLES:** 24,998
- **DEC_INSTRS:** 16,664

4.5 Summary Table (All Results)

Core	Language	ENC Cycles	DEC Cycles	ENC Instr.	DEC Instr.
PicoRV32	C	52,105	50,958	9,267	9,273
PicoRV32	Rust	99,224	100,182	16,488	16,637
VexRiscv	C	10,834	10,764	9,267	9,273
VexRiscv	Rust	24,765	24,998	16,515	16,664

---

5. Analysis

5.1 VexRiscv vs PicoRV32

- **VexRiscv is ~4–5× faster than PicoRV32** for both encryption and decryption.
- This is due to:
  - Instruction pipelining in VexRiscv
  - More efficient arithmetic datapath
  - Lower CPI (cycles per instruction)

PicoRV32, lacking a pipeline, executes each instruction in one long cycle, making heavy ARX operations slower.

5.2 C vs Rust

From measurements:

- Rust produces **~2× more instructions** than C
- Rust takes **~2× more cycles** than C on both soft-cores
- This matches the behavior expected from:
  - Heavier calling conventions
  - LLVM backend prioritizing safety over minimal code size
  - Larger register usage and stack frames

However, Rust still performs correctly and predictably across all platforms.

5.3 Observations on SPARKLE Performance

- SPARKLE’s ARX operations map efficiently to RISC-V integer pipelines
- The algorithm is predictable, with clear correlation between cycle count and number of SPARKLE rounds
- Good suitability for FPGA soft-cores, especially VexRiscv

---

6. Conclusions

From the benchmarking study:

1. **Schwaemm128 is efficiently implementable on RISC-V soft-cores**, even minimal ones like PicoRV32.
2. **VexRiscv delivers significantly better performance**, running ~4–5× faster due to pipelining and architectural optimizations.
3. **Rust introduces roughly double the instruction count and runtime**, consistent with both compiler overhead and code generation choices.
4. **Rust cross-compilation took ~2× longer than C**, though this does not affect runtime performance.
5. **Vivado provided a stable platform** for synthesizing and implementing the CPUs and peripherals.

Overall, Schwaemm128 demonstrates strong practical performance for lightweight security on FPGA-based RISC-V systems. VexRiscv is clearly the superior soft-core for cryptographic workloads, while PicoRV32 remains a viable choice when area minimization is more important than performance.
